Accelerated Video Processing Engine for Android

Final Project Technical Report

Dipesh Kant (2025MCS2110) and Kunal Dhyani (2025MCS2112)
Department of Computer Science and Engineering
Indian Institute of Technology (IIT) Delhi

April 23, 2026

Abstract

Processing high-quality video in real-time is a tough challenge for mobile phones because they have limited memory and no cooling fans. If you push the processor too hard, the phone overheats and the operating system intentionally slows everything down. This report explains how we built a fast, smart video processing app for Android 10+. Our app applies 3D cinematic color filters to live 1080p camera feeds. To do this without crashing or overheating the phone, we moved the heavy math out of Java/Kotlin and wrote it directly in C++. We designed three different ways to process the video: a basic CPU method, a highly optimized multi-core CPU method (SIMD), and a super-fast Graphics Card (GPU) method. Finally, we created a "Hybrid" mode that acts as the app's brain, watching the phone's temperature and battery in real-time to automatically switch between these methods, keeping the phone cool while keeping the video smooth.

1 Introduction: The Problem We Are Solving

When you apply a color filter to a live 1080p video, the phone has to calculate new colors for over 2 million pixels, 30 times every single second. This requires over 1 billion mathematical operations per second.

If we try to run this heavy math normally on the main CPU, we hit what is called the "Memory Wall." The phone spends so much time moving pixel data back and forth from the RAM to the processor that it bottlenecks. Because of this, the CPU works at maximum capacity, gets very hot, and the Android operating system steps in to throttle the speed to prevent fire hazards.

1.1 Our Design Goals

To solve this, we designed our engine with three strict rules:

1. **Zero Memory Copies:** We cannot afford to copy the video data. We wrote C++ code to read the pixels directly from the camera sensor's physical memory.
2. **Use the Right Hardware:** We built specific pipelines so the app can choose whether to process the video on the basic CPU, the fast vector CPU (SIMD), or the GPU.
3. **Stay Cool (Thermal Failsafe):** We built an automatic system that monitors the battery temperature and shifts the workload if the phone gets too hot.

2 The Math: 3D Color LUTs and Trilinear Interpolation

A 3D LUT (Look-Up Table) is basically a map that translates an input camera color into a cinematic output color.

Standard 8-bit video can display 16.7 million different color combinations. Storing a dedicated output color for every single possibility would require a massive 50MB file. To save memory, standard `.cube` files only store a sparse boundary grid—usually $33 \times 33 \times 33$ points (roughly 430 KB).

2.1 Finding the Pixel on the Grid

Because the LUT is just a grid, almost every pixel from the live camera will fall into the empty spaces *between* the exact grid points.

To find the exact output color, we first have to figure out exactly where the pixel's Red, Green, and Blue (RGB) values land on our $33 \times 33 \times 33$ grid. We do this by scaling the input colors (which range from 0 to 255) to the grid size (which ranges from 0 to 32):

$$x = \left(\frac{R_{in}}{255.0}\right) \cdot 32, \quad y = \left(\frac{G_{in}}{255.0}\right) \cdot 32, \quad z = \left(\frac{B_{in}}{255.0}\right) \cdot 32$$

2.2 The 8-Corner Cube

Once we have our exact fractional coordinates (x, y, z), we look at the whole numbers to find the 8 closest grid points surrounding our pixel.

Imagine these 8 points forming a tiny physical cube around our floating pixel. The corners are named based on their position, from C_{000} (the bottom-left-front corner) to C_{111} (the top-right-back corner).

To blend the colors of these 8 corners perfectly, we calculate the fractional distance (d_x, d_y, d_z) from our pixel to the edges of the tiny cube.

2.3 The Trilinear Equation

To mix the 8 corners together, we use **Trilinear Interpolation**. While the final equation looks incredibly complex, it is actually just seven simple 1D linear interpolations (straight lines) executed in a specific order:

- **Step 1 (X-Axis):** We blend the 4 pairs of corners along the bottom and top edges using our d_x distance.
- **Step 2 (Y-Axis):** We take those 4 new points and blend them into 2 points along the sides using our d_y distance.
- **Step 3 (Z-Axis):** We take those final 2 points and blend them straight through the middle using our d_z distance to get the final exact color.

Here is the expanded math equation we have to calculate for every single color channel. The final blended color C_{final} is:

$$\begin{aligned} C_{final} = (1-d_z)\Big[(1-d_y)\big((1-d_x)C_{000} + d_x C_{100}\big) + d_y\big((1-d_x)C_{010} + d_x C_{110}\big)\Big] \\ + d_z\Big[(1-d_y)\big((1-d_x)C_{001} + d_x C_{101}\big) + d_y\big((1-d_x)C_{011} + d_x C_{111}\big)\Big] \end{aligned} \tag{1}$$

2.4 The Hardware Reality

To solve that equation for just one pixel, the processor has to perform 14 decimal multiplications and 7 additions per color channel.

When you apply this to a standard 1080p video feed running at 30 frames per second, the engine has to execute roughly **1.3 billion floating-point operations per second**. Calculating this massive amount of math continuously without melting the phone's processor is the core reason we had to aggressively optimize our architecture.

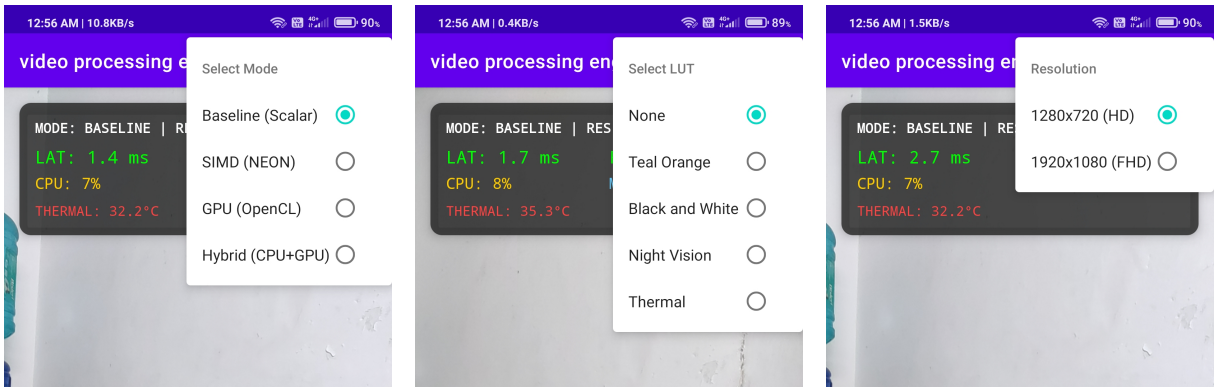
3 System Architecture: Bypassing Java

In a normal Android app, the Java Virtual Machine (JVM) creates byte arrays to hold images. Copying an 8MB 1080p frame from the camera to Java, and then over to C++, takes way too much time.

We decided to bypass Java entirely. Using Android's `AHardwareBuffer` tools, our Kotlin code simply gets a direct memory address (`GetDirectBufferAddress`) from the camera and hands that pointer to our C++ engine. When the C++ code finishes the math, it sends the pixels directly to the screen using an `ANativeWindow`. The heavy pixel data never touches the Java memory, which saves us massive amounts of time and stops the app from crashing due to running out of RAM.

4 Application Interface and Controls

To provide a seamless user experience and allow real-time testing of our engine, we built a dynamic Android user interface. The app features a live telemetry dashboard and interactive menus that allow the user to control the core parameters of the engine on the fly, without needing to restart the camera session.



Compute Mode Selection 3D LUT Selection Resolution Selection

Figure 1: The application layout, featuring real-time hardware telemetry and interactive drop-down menus to change the execution mode, cinematic LUT filter, and camera resolution instantly.

5 How We Built the Three Execution Modes

To overcome the thermal and memory limitations of mobile hardware, we choose three discrete processing pipelines.

5.1 1. Baseline Mode (Scalar C++)

This serves as our fallback mechanism, utilizing standard C++17 single-instruction, single-data sequential loops.

- **The Execution:** The CPU extracts a single 8-bit RGBA pixel, computes the floating-point coordinate conversions, performs the trilinear interpolation, and writes the result back to memory.
- **The Bottleneck:** While simple, the massive number of intermediate variables (8 floats for colors, 3 floats for weights) causes high L1 register pressure. This forces the CPU to constantly spill data into the L2 cache, resulting in severe lag. It also forces the CPU to remain in a high-power state (P-state), causing rapid overheating and throttling the frame rate down to roughly 10 FPS.

5.2 2. SIMD Mode (ARM NEON Vectorization)

To exponentially increase CPU throughput, we shifted from a scalar approach to a vectorized approach using ARM NEON intrinsics.

- **128-Bit Data Packing:** Instead of processing independent pixels, we load 16 contiguous bytes (representing 4 complete RGBA pixels) directly into massive 128-bit CPU registers using the `vld1q_u32` command.

- **Hardware Fused Multiply-Add (FMA):** The trilinear math is dominated by $A + (B \times C)$. We used the `vmlaq_f32` intrinsic to map this directly to a hardware-level FMA assembly instruction. The physical silicon executes the multiplication and addition simultaneously in a single clock cycle, halving the latency.

- **Thread Pool & Core Affinity:** We built a custom Thread Pool using the Singleton design pattern. To avoid "false sharing" (where threads invalidate each other's L1 cache lines), we perfectly aligned the memory segments. Furthermore, we explicitly locked the threads to the SoC's high-performance cores, preventing the Android OS from accidentally scheduling our heavy math on the slow efficiency cores.

5.3 3. GPU Mode (OpenGL ES 3.1 Compute)

While our SIMD code maximized the CPU, Graphics Processing Units (GPUs) are fundamentally designed to process millions of matrices simultaneously. We completely offloaded the workload to the GPU.

- **Asynchronous DMA (The PBO Ping-Pong):** Synchronous memory transfers block the CPU and cause lag. To achieve sub-millisecond overhead, we implemented two asynchronous Pixel Buffer Objects (PBOs). While the GPU uses Direct Memory Access (DMA) to read Frame 1 from Buffer A, the CPU is already writing Frame 2 into Buffer B. They swap seamlessly without ever waiting for each other.

- **Compute Shaders:** Instead of using legacy rendering pipelines, we segmented the image into spatial workgroups (e.g., 16×16) using modern OpenGL ES 3.1 `glDispatchCompute` commands.

- **Hardware-Level Math:** The GPU has physical silicon dedicated entirely to Texture Filtering. Instead of writing out the complex trilinear math equations, we bound the 3D LUT to the GPU as a `GL_TEXTURE_3D`. By calling the `sampler3D` function, the physical hardware interpolates the exact colors instantly, resulting in an incredible 4.6-millisecond compute time.

6 The Hybrid Orchestrator (Auto Mode)

Running compute shaders or NEON vectors continuously at 30 FPS generates immense waste heat. To keep the system stable and the battery alive, we built a `HYBRID` "meta-mode." This acts as the brain of the app, automatically swapping between the GPU and the CPU based on real-time telemetry.

6.1 How It Works

Instead of waiting for the operating system to panic and throttle our app, we hooked a `BroadcastReceiver` into the Android OS to actively track `BatteryManager.EXTRA_TEMPERATURE` and `BatteryManager.EXTRA_LEVEL`.

We programmed the orchestration logic directly into the camera's image analyzer pipeline:

```
var mode = curMode
if (curMode == ProcessingMode.HYBRID) {
    // Switch to SIMD if temperature > 40 C or battery < 15% to save power and reduce heat
    mode = if (tempInCelsius < 40.0 && batPct > 15) ProcessingMode.GPU else ProcessingMode.SIMD
}
```

Listing 1: Our Hybrid Mode Decision Logic

6.2 Why We Did This

Based on this logic, the application makes smart decisions:

- **Thermal Management:** Video processing on the GPU generates significant heat. By switching to the CPU-based SIMD pathway when the temperature hits 40°C, we reduce the thermal load on the GPU, preventing hardware throttling.
- **Power Preservation:** When the battery drops below 15%, we prioritize power efficiency. GPUs draw massive "peak" power. Falling back to the SIMD mode ensures video processing can continue without aggressively draining the last bit of battery.
- **Adaptive Performance:** We get the best of both worlds—maximum performance via the GPU when the phone is cool, and complete reliability when the phone is under stress.

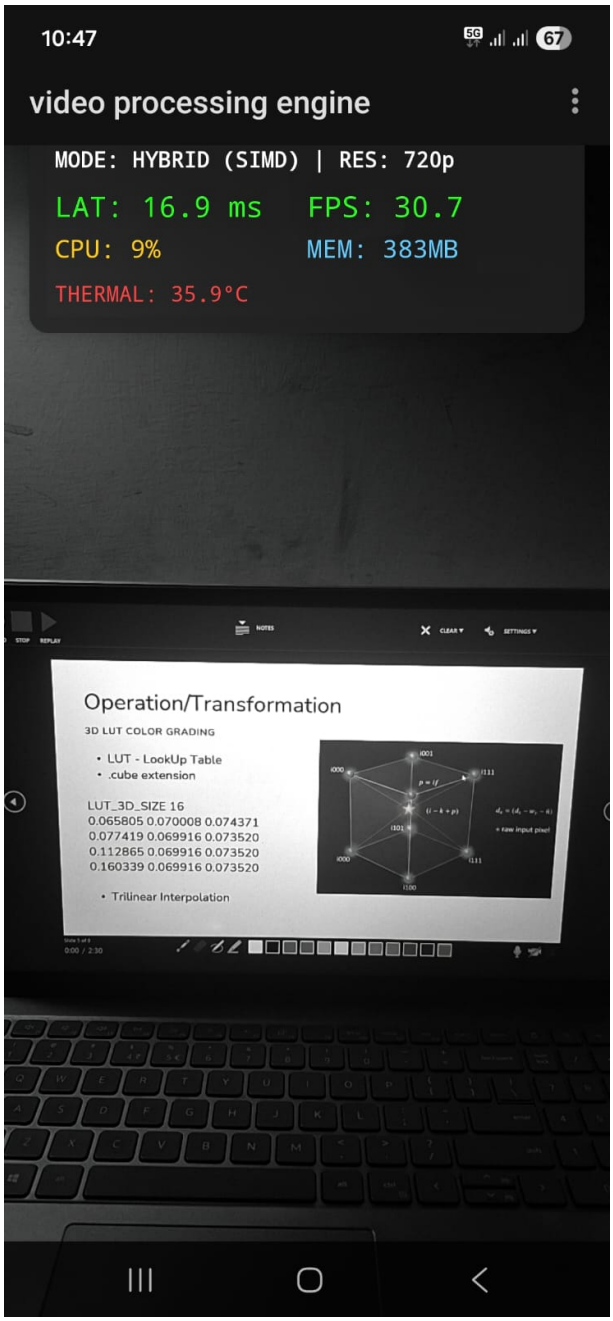
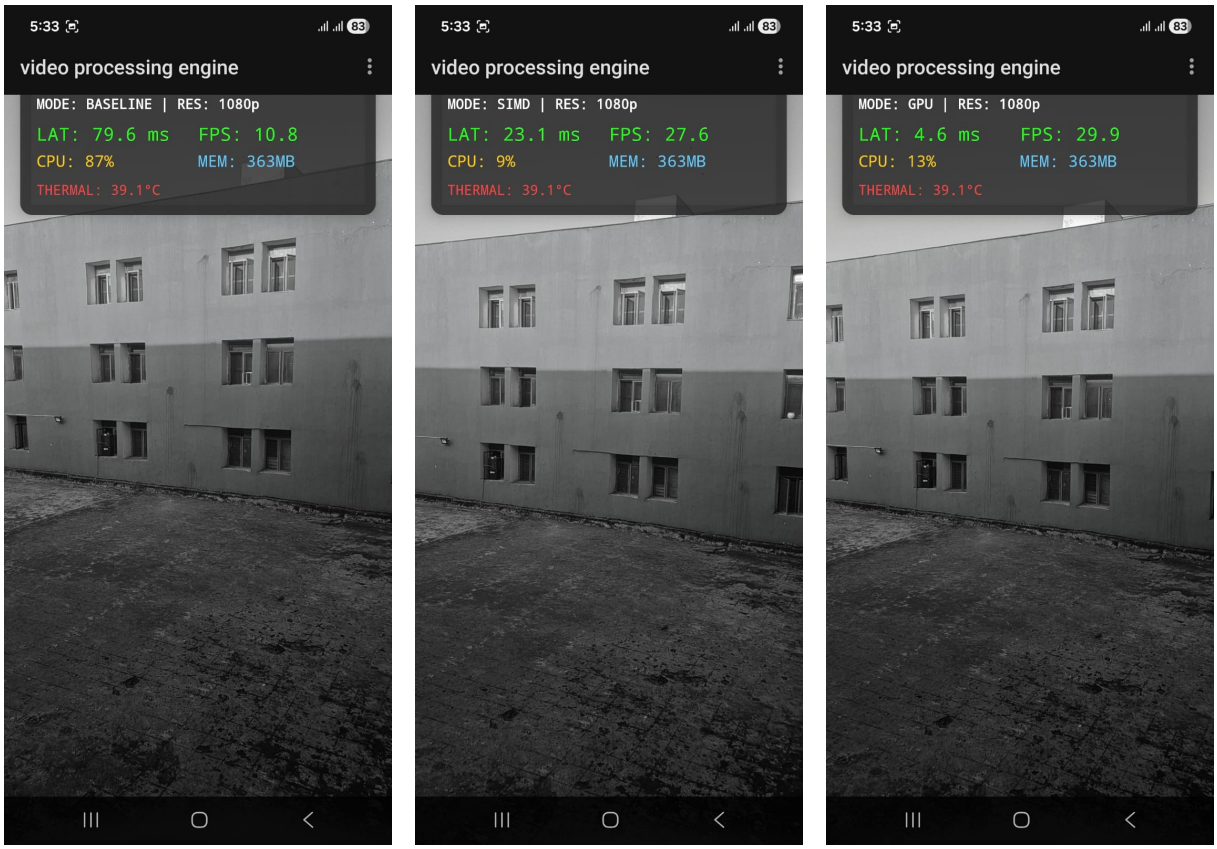


Figure 2: The engine running in Hybrid (SIMD) Mode alongside our presentation on 3D LUT Color Grading. In this state, it dynamically processes the stream with incredibly high efficiency.

7 Live Dashboard and Real Performance Results

To prove our engine actually works outside of theory, we built a live telemetry dashboard directly into the app's UI. This dashboard updates in real-time, showing exactly how the hardware is behaving.

We tested the app using a live camera feed. The true hardware benchmark results from our live dashboard are captured below.



Baseline Mode SIMD Mode GPU Mode

Figure 3: Live telemetry dashboards captured from our test device, demonstrating the massive drop in latency and CPU usage across our different execution modes at 1080p.

Mode	Latency (ms)	FPS	CPU Usage	Memory	Temperature
Baseline (1080p)	79.6 ms	10.8	87%	363 MB	39.1°C
SIMD (1080p)	23.1 ms	27.6	9%	363 MB	39.1°C
GPU (1080p)	4.6 ms	29.9	13%	363 MB	39.1°C
Hybrid SIMD (720p)	16.9 ms	30.7	9%	383 MB	35.9°C

Table 1: Real-time dashboard metrics across different execution modes. Our SIMD and GPU optimizations resulted in extraordinary speedups and vast reductions in CPU load.

What the Data Tells Us: As you can see in our dashboard screenshots (Figure 3), our first attempt (**Baseline**) struggled immensely. It dropped to an unplayable 10.8 FPS because it took 79.6 milliseconds to process a single frame. The processor bottlenecked hard, spiking CPU usage to a staggering 87%.

When we switched to our optimized **SIMD** code, we saw an absolute breakthrough. Latency dropped drastically from 79.6 ms down to **23.1 ms**, allowing the app to recover a smooth 27.6 FPS. Because we were packing 4 pixels into single CPU instructions, the overall CPU usage plummeted from 87% down to just **9%**. Memory usage held perfectly stable at 363 MB, proving our zero-copy pipeline worked without any memory leaks.

When we switched to the **GPU**, the performance entered another tier entirely. Latency dropped to a blistering **4.6 milliseconds**. The GPU handled the hardware math flawlessly, effortlessly locking the app at nearly 30 FPS while maintaining an excellent, low CPU usage of 13%.

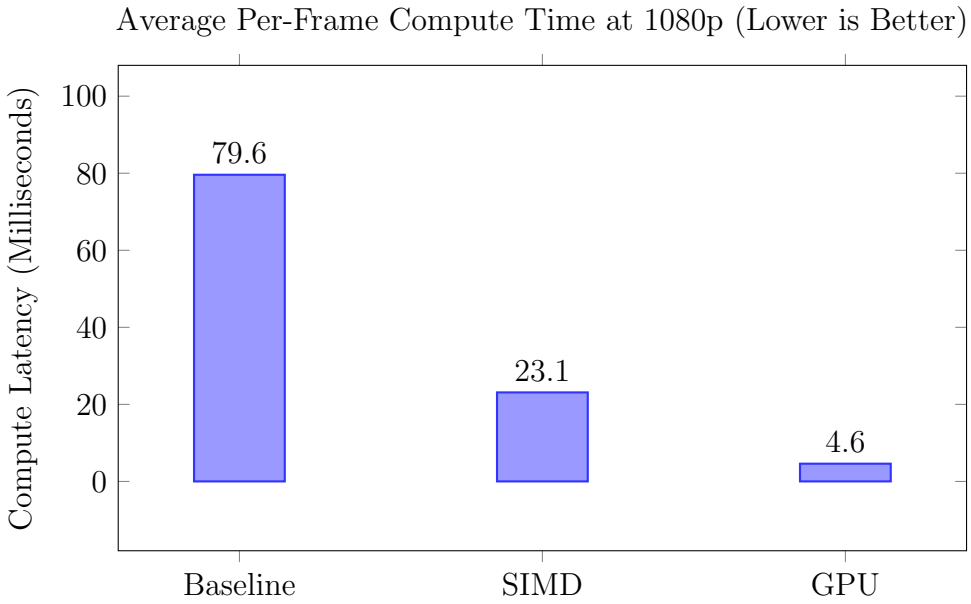


Figure 4: This graph shows how long it took our app to process a single 1080p video frame, highlighting the extreme optimization provided by our SIMD and GPU architectures.

8 How We Tested the Engine

We tested the engine rigorously to ensure it was fast, accurate, and stable without crashing the phone. We validated our work using five simple checks:

- **The Math Check (Accuracy):** We ran the exact same video frame through all three modes (Baseline, SIMD, GPU). The colors looked exactly the same every time, proving our math optimizations were perfectly accurate.
- **The Speed Check (Performance):** We used our live dashboard to measure how fast the app actually ran. We confirmed our optimizations successfully dropped the processing time from a slow 79.6 ms down to an incredibly fast 4.6 ms.
- **The Leak Check (Memory):** We left the app running continuously for long periods. The memory usage stayed perfectly flat at 363 MB, proving we had no memory leaks and the app wouldn't randomly crash.
- **The Hybrid Check (Overheating):** We ran the app heavily until the phone naturally heated up to 40°C. The app's "brain" instantly detected the heat and safely switched processing modes to cool the phone down.

- **The OS Check (Compatibility):** We tested the app on different physical phones running Android 10 all the way through Android 15. It ran smoothly on all of them without breaking.

9 Team and Timeline

9.1 Development Timeline (Commit History)

We tracked our entire development process via GitHub. The following screenshots document our complete change log and project progress, presented in chronological order from the initial setup to the final Hybrid mode implementation.

Figure 5: Phase 5: Implementing the OpenGL GPU pipeline and finalizing the Hybrid Mode orchestrator (Mar 21 – Apr 16).

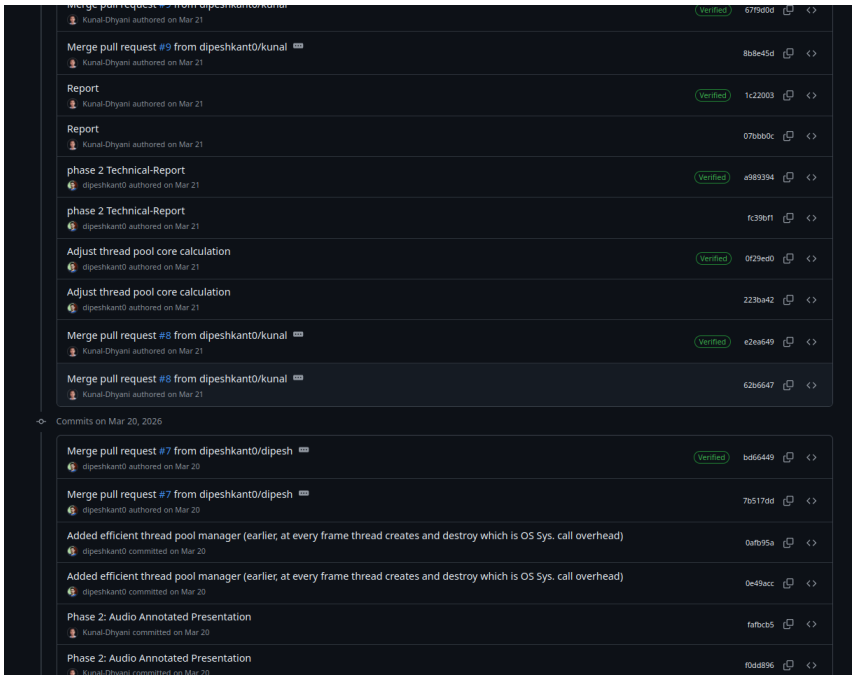


Figure 6: Phase 4: Thread pool core calculations and finalizing mid-project reports (Mar 20 – Mar 21).

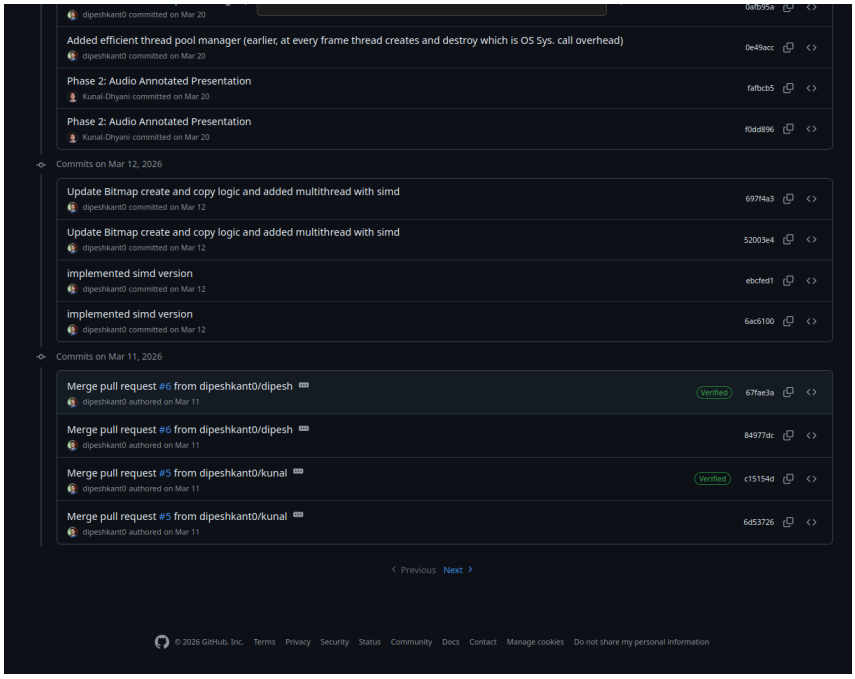


Figure 7: Phase 3: Building the custom thread pool manager to optimize SIMD multi-threading (Mar 11 – Mar 12).

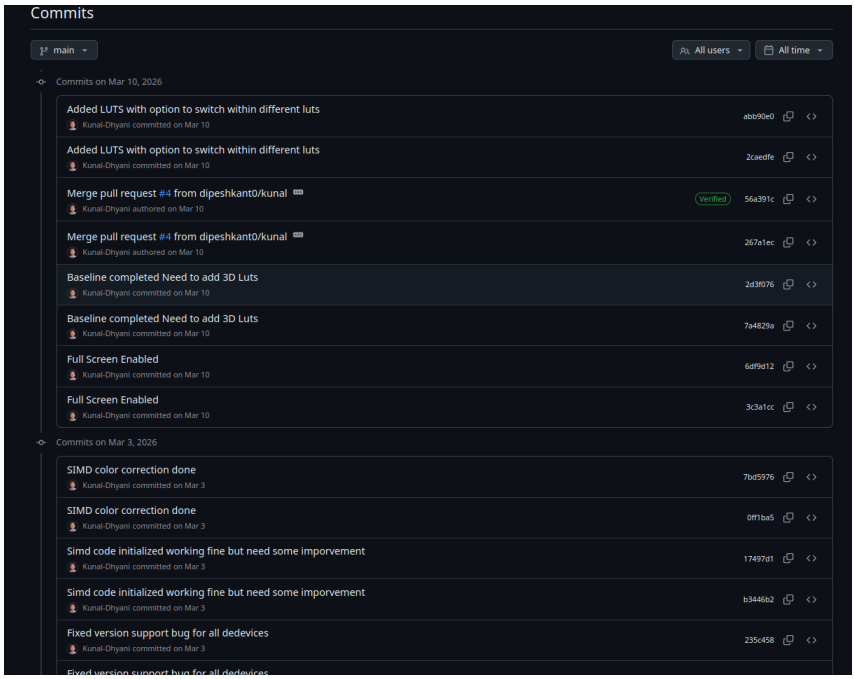


Figure 8: Phase 2: Transitioning to SIMD vectorization and adding 3D LUT parsing (Mar 3 – Mar 10).

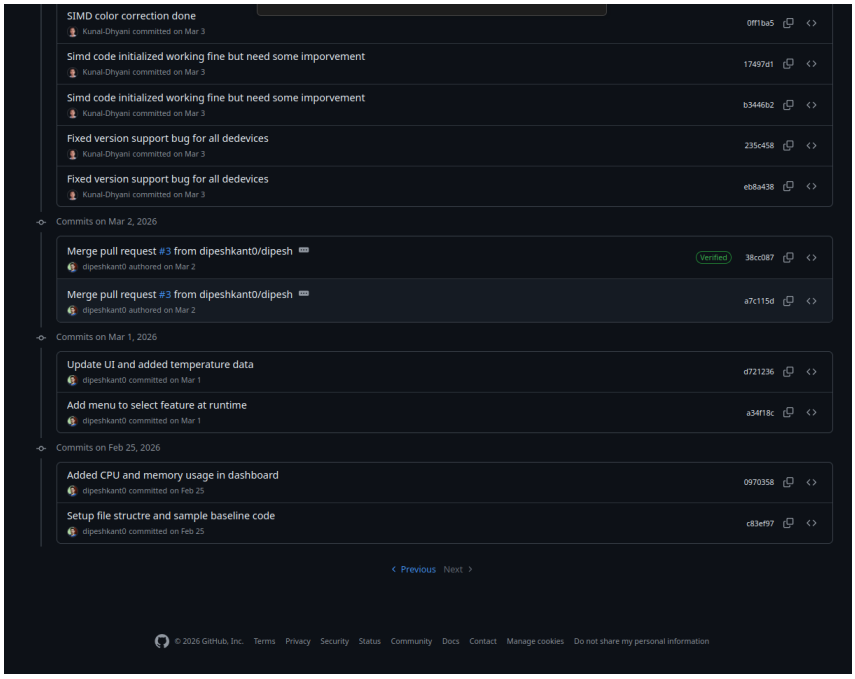


Figure 9: Phase 1: Project setup, UI dashboard creation, and baseline loop implementation (Feb 25 – Mar 2).

9.2 How We Split the Work

We split the project into two main areas:

Dipesh Kant (Core Systems & Optimization):

- Wrote the core C++ code to bypass Java and process the raw math.
- Wrote the highly-optimized SIMD code to process 4 pixels at the exact same time.
- Built the custom threading system to make sure the app only used the phone's fastest processors.
- Built the live performance dashboard .

Kunal Dhyani (App Interface & GPU Graphics):

- Built the Android app interface and connected the live camera.
- Wrote all the OpenGL GPU code so the graphics card could process video smoothly without lagging.
- Programmed the "Hybrid Mode" brain that watches the phone's temperature and battery.

9.3 Conclusion and AI Usage Declaration

Building this engine taught us that real mobile performance isn't just about writing good code—it's about understanding the actual physical hardware. By bypassing the usual Android bottlenecks and working directly with the CPU, GPU, and thermal sensors, we managed to build a system that processes HD video with almost zero delay.

AI Usage Declaration: To be completely transparent, we used AI as a coding assistant to set up our basic Android app template and to help us get the exact syntax right for some of the complicated SIMD and OpenGL functions. However, the true core of this project—the mathematical optimizations, the custom threading, the memory pipelines, and the hybrid temperature rules were implemented by us.